

On-Device Conversational AI with a Post-Trained Qwen3.5 Model on Android

Submitted in partial fulfillment of the requirements of the degree

BACHELOR OF ENGINEERING

IN

ARTIFICIAL INTELLIGENCE (AI) & DATA SCIENCE

By

Rohit Swami Roll No 1

Namish Sharma Roll No 2

Akhilesh Pottekat Roll No 3

Supervisor

Prof. / DR. guide name



**Department of Artificial Intelligence (AI) & Data Science
SIES Graduate School of Technology (Autonomous)**

Affiliated to University of Mumbai

Nerul, Navi Mumbai – 400706

A.Y. 2026–27

CERTIFICATE

This is to certify that the project entitled “**On-Device Conversational AI with a Post-Trained Qwen3.5 Model on Android**” is a bonafide work of the following students, submitted to the SIES Graduate School of Technology in partial fulfillment of the requirement for the award of the degree of **Bachelor of Engineering** in “**Artificial Intelligence (AI) & Data Science**”.

Rohit Swami	Roll No 1
Namish Sharma	Roll No 2
Akhilesh Pottekat	Roll No 3

Dr. Arathi Boyannapalli
Supervisor

Dr. Neethu Anna Sabu
Mini Project Coordinator

Dr. Arathi Boyannapalli
Head of Department

Dr. K. Lakshmisudha
Principal

PROJECT REPORT APPROVAL

This Mini Project entitled “ **On-Device Conversational AI with a Post-Trained Qwen3.5 Model on Android** ” by following students is approved for the degree of **Bachelor of Engineering in Artificial Intelligence (AI) and Data Science**.

Rohit Swami	Roll No 1
Namish Sharma	Roll No 2
Akhilesh Pottekat	Roll No 3

Examiners

1.....

(Internal Examiner Name & Sign)

2.....

(External Examiner Name & Sign)

Date:

Place: SIES GST, Nerul, Navi Mumbai.

ACKNOWLEDGEMENT

We would like to express our thanks to the people who have helped us the most throughout our project. We are grateful to our guide (**Dr. Arathi Boyannapalli**) and coordinator, **Prof. Akshay A Jadhav** for nonstop support for the project.

A special thanks goes to each other who worked together as a team in completing the project, where we all exchanged our own interesting ideas and thoughts and made it possible to complete our project with all accurate information. We also wish to thank our parents for their personal support and attention, who inspired us to go our own way.

We would also like to extend our sincere gratitude to our Principal, **Dr. K. Lakshmisudha**, and our Head of the Department, **Dr. Arathi Boyannapalli**, for their continuous support and encouragement.

We would also like to thank our other faculty members for providing us with all the required resources and references for the project.

Project Team

Rohit Swami

Namish Sharma

Akhilesh Pottekat

ABSTRACT

Large language models are typically served from the cloud, which raises privacy, cost, and connectivity concerns for everyday mobile use. This project demonstrates a fully on-device conversational assistant on Android, powered by a domain post-trained Qwen3.5-4B model. Two from-scratch pretraining attempts came first—a 68.7M dense model that finished training yet could not produce coherent answers, and a 200M mixture-of-experts model that stalled at $\approx 13\%$ of its step budget under Kaggle time/quota limits—forcing the switch to post-training. A QLoRA fact-checking adapter was first trained on mixed grounded science and mathematics data, continued with replay, and finished with reference-free ORPO preference training (final ORPO-v4 adapter). The adapter was merged into the base weights and converted to a 2.5 GB Q4_K_M GGUF file so it runs with llama.cpp on a phone. An Android application built on the llama.cpp inference binding auto-loads the post-trained model, renders the model’s `<think>` reasoning in expandable thought cards, executes `web_search` tool calls through an in-app search tool with a strict parse–execute–respond loop, and ships a ChatGPT-style light/dark chat interface. On screening benchmarks the ORPO-v4 adapter improved the selected-metric average by +8.07 percentage points over the base model (GSM8K 0% to 60% at $n=10$; 6% to 66% on a paired $n=50$ check), and the quantized model was verified live on an Android emulator, answering questions and completing web-search tool calls end to end.

Keywords: On-Device LLM, Qwen3.5, QLoRA, ORPO, GGUF Quantization, llama.cpp, Android, Tool Calling, Web Search

Contents

ACKNOWLEDGEMENT	iii
ABSTRACT	iv
List of Figures	v
List of Tables	vi
List of Abbreviations	vii
1 Introduction	1
1.1 Overview	1
1.2 Motivation	1
1.3 Background and Project Evolution	2
1.4 Problem Statement and Objectives	2
1.4.1 Problem Statement:	2
1.4.2 Objectives:	3
1.5 Scope	3
1.6 Organization of the Report	4
2 Literature Survey	5
2.1 Survey of Existing System	5
2.1.1 Comparative Analysis of Existing Systems	6
2.2 Limitations of Existing System	6
3 Proposed System	8
3.1 System Overview	8
3.2 Components of the System	8
3.3 Architecture/Framework:	9

3.4	Algorithm and Process Design:	9
3.5	Details of Hardware & Software	10
4	Design and Methodology	11
4.1	Design Details	11
4.1.1	Entity-Relationship (ER) Diagram	11
4.1.1.1	Key Entities and Their Attributes	11
4.1.1.2	Relationships Between Entities	13
4.1.1.3	Design Advantages	13
4.1.2	UML Activity Diagram	13
4.1.2.1	Activity Flow	14
4.1.2.2	Features of the Activity Design	14
4.1.3	Data Flow Diagram (DFD) – Level 1	14
4.1.3.1	External Entity: User	15
4.1.3.2	Core Processes	15
4.1.3.3	Data Stores	16
4.1.3.4	External System: Web Search APIs	16
4.1.3.5	Workflow Summary	16
4.1.3.6	Design Benefits	16
4.2	Methodology	16
4.3	Algorithm Implementation	17
4.4	Details of Hardware & Software	17
4.4.1	Hardware Requirements	17
4.4.2	Software Requirements	17
5	Results and Discussions	18
5.1	Implementation	18
5.1.1	User Interface Overview	18
5.1.2	Key Features	18
5.1.3	Model Loading	18
5.1.4	Think/Tool Loop	18
5.1.5	Query and Response Generation	19
5.2	Results	19

5.2.1 Training and Benchmark History	19
6 Conclusion and Future Scope	24
Plagiarism Report	26
Publication Details / Project Achievements	27
Certificates	28

List of Figures

Figure No.	Figure Caption	Page No.
1.1	Project Schedule – Gantt Chart	2
1.2	Project evolution: pretraining to post-training	3
3.1	On-device system architecture	8
4.1	ER Diagram	10
4.2	UML Activity Diagram	13
4.3	DFD Level 1	15
5.1	App home screen (light mode)	21
5.2	Dark-mode chat with web-search tool card	22
5.3	Benchmark screening: Base vs ORPO-v4	24
5.4	Paired GSM8K/IFEval check (n=50)	24

List of Tables

Table No.	Table Caption	Page No.
2.1	Comparative analysis of on-device LLM systems	5
5.1	Training history across project phases	23
5.2	Seven-task screening: Base vs ORPO-v4	23
5.3	On-device deployment measurements	25

List of Abbreviations

API	Application Programming Interface
CPU	Central Processing Unit
DFD	Data Flow Diagram
ER	Entity Relationship
GGUF	GPT-Generated Unified Format (single-file model format)
GPU	Graphics Processing Unit
JNI	Java Native Interface
LLM	Large Language Model
LoRA	Low-Rank Adaptation
NDK	Native Development Kit
NLP	Natural Language Processing
ORPO	Odds-Ratio Preference Optimization
QLoRA	Quantized Low-Rank Adaptation
RAG	Retrieval-Augmented Generation
SDK	Software Development Kit
UI	User Interface
UML	Unified Modeling Language

Chapter 1

Introduction

1.1 Overview

This project builds a conversational AI assistant that runs entirely on an Android phone, with no cloud dependency at inference time. The brain of the app is Qwen3.5-4B, post-trained in three stages—QLoRA supervised fine-tuning, replay continuation, and reference-free ORPO preference training—to improve grounded, evidence-aware answering while preserving mathematics and reasoning ability. The final ORPO-v4 adapter is merged into the base weights and quantized to a 2.5 GB Q4_K_M GGUF file that executes through llama.cpp on-device. A native Android application auto-loads this post-trained model, displays the model’s `<think>` reasoning traces, and resolves `web_search` tool calls with an in-app search tool, all inside a ChatGPT-style chat interface with light and dark modes. The project did not start here: two from-scratch pretraining attempts (a 68.7M dense model and a 200M mixture-of-experts model on free Kaggle GPUs) are documented in Section 1.3, and the switch to post-training an existing model was forced by training time, quota limits, and answer quality.

1.2 Motivation

Cloud-hosted assistants send every keystroke to a remote server. That costs money, fails without connectivity, and leaks private conversations. At the same time, post-training techniques such as LoRA and ORPO make it practical for students to specialize open models, and runtimes such as llama.cpp make it practical to run them on a phone. The motivation of this work is to join those two advances: take a model the team post-trained itself, compress it to phone size, and ship a genuinely usable mobile assistant around it.

1.3 Background and Project Evolution

The project evolved through three phases (Figure 1.1). **Phase 1** pretrained a 68.7M dense Transformer from scratch on Kaggle T4 GPUs: 50,000 steps ($\approx 3.28\text{B}$ tokens, ≈ 10 epochs), finishing at validation loss 4.5870 (perplexity ≈ 98.2 , accuracy $\approx 26\%$). The run was stable, but sampled generations were short fragments with no long-range coherence or factual recall—the verdict of the evaluation harness was blunt: 68M parameters are not enough for usable answers. **Phase 2** scaled up to a $\approx 200\text{M}$ mixture-of-experts model (181M total / 105M active, Muon optimizer) with a 26,300-step budget. Training progressed (loss 10.34 $\rightarrow \approx 5.35$, accuracy 2.3% $\rightarrow \approx 14\%$) but far too slowly: $\approx 500\text{--}800$ tokens/s on T4s, 12-hour Kaggle session walls that repeatedly discarded unsaved steps, exhausted 30 h/week GPU quotas across accounts, dead P100 hardware (unsupported by modern PyTorch), and host-RAM kills on TPU slices. By early September the run stood at step 3,500 of 26,300 ($\approx 13\%$) with weeks of grinding still ahead. **Phase 3** therefore switched strategy under the time constraint: instead of pretraining a small model further, post-train an existing strong model (Qwen3.5-4B) with QLoRA, replay, and ORPO—which reached a usable, benchmark-verified assistant in days rather than months, and is the system reported in the remaining chapters.

Project evolution: pretraining to post-training

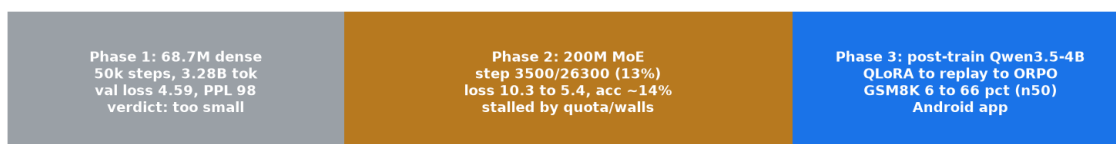


Figure 1.1: Project evolution: pretraining to post-training

1.4 Problem Statement and Objectives

1.4.1 Problem Statement:

How can a 4-billion-parameter post-trained language model with tool-calling behaviour be deployed, invoked, and presented inside a responsive Android application—offline, with visible

reasoning and working web-search tool use—without cloud inference?

1.4.2 Objectives:

1. Post-train Qwen3.5-4B with QLoRA + replay + ORPO for grounded, evidence-aware answers.
2. Merge the adapter and convert it to an Android-loadable Q4_K_M GGUF artifact.
3. Build an Android Studio application on the llama.cpp inference binding that auto-loads the post-trained model.
4. Implement a strict think/tool-call parse–execute–respond loop with an in-app web_search tool, so raw tool markup is never shown.
5. Deliver a polished ChatGPT-style chat UI with thinking cards, tool cards, and light/dark modes.
6. Evaluate screening benchmarks and verify the full loop live on an Android emulator.

1.5 Scope

- Post-training Qwen3.5-4B with QLoRA ($r=32$), replay continuation, and ORPO preference training on grounded science, mathematics, and anti-hallucination data.
- Merging the LoRA adapter and exporting F16 and Q4_K_M GGUF artifacts with llama.cpp conversion tooling.
- An Android (minSdk 33) chat application: auto model loading, streaming chat, thinking cards, tool cards, web search via public instant-answer APIs, and a DayNight theme.
- Screening evaluation (7 tasks at $n=10$, paired GSM8K/IFEval at $n=50$) plus live on-device verification on an emulator.
- Out of scope: training a foundation model from scratch, full-scale benchmark replication, and Play Store release hardening.

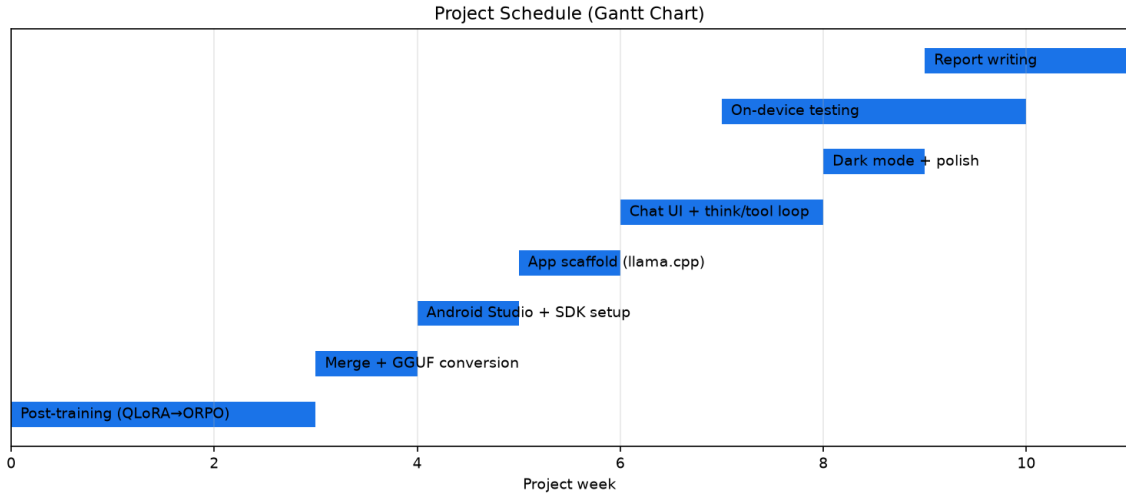


Figure 1.2: Project Schedule – Gantt Chart

1.6 Organization of the Report

Chapter 1 **Introduction** presents the overview, motivation, problem statement, and scope. Chapter 2 **Literature Survey** reviews on-device runtimes, efficient fine-tuning, and tool calling. Chapter 3 **Proposed system** describes the architecture and components. Chapter 4 **Design and Methodology** gives the ER, UML, and DFD designs with the implementation methodology. Chapter 5 **Results and Discussions** shows the app, the tool loop, and the evaluation. Chapter 6 concludes with future scope.

Chapter 2

Literature Survey

Running large language models on phones requires progress on three fronts: efficient inference runtimes, aggressive quantization, and parameter-efficient post-training. This survey reviews the systems and techniques combined in this project, compares on-device options, and notes their limitations.

2.1 Survey of Existing System

llama.cpp [1] is the de-facto C/C++ inference engine for local LLMs: it introduced the GGUF single-file format, K-quant quantization families (including Q4_K_M used here), and CPU backends with hardware-specific kernels, plus an Android binding example. Google’s MediaPipe LLM Inference API [2] offers GPU-accelerated on-device inference but targets Gemma-family models and a fixed task API, which fits less well with a custom post-trained Qwen artifact. MLC LLM [3] compiles models with Apache TVM for broad hardware coverage at the cost of a heavier build pipeline, while Ollama [4] focuses on desktop/server serving rather than mobile apps. For post-training, LoRA [5] freezes base weights and trains low-rank adapters; QLoRA [6] adds 4-bit quantization so a 4B model fits a 12 GB GPU; ORPO [7] aligns models with preferences without a reference model. Tool-calling assistants build on the same chat-template conventions (Qwen tool templates) this project relies on for `web_search`.

2.1.1 Comparative Analysis of Existing Systems

Sr. No.	System / Ref No.	Year	Method Used	Key Findings
1	llama.cpp [1]	2023–26	GGUF format, K-quants, CPU/ARM kernels, Android example	Runs 4B models on phones; Q4_K_M $\approx$ 2.5 GB; used as our runtime
2	MediaPipe LLM Inference [2]	2024	GPU delegates, task API	Fast but model/task restricted; not used
3	MLC LLM [3]	2023	TVM compilation, GPU/NPU	Broad hardware; heavier pipeline; not used
4	Ollama [4]	2023	llama.cpp-based server	Desktop/server serving; no mobile UI
5	LoRA [5]	2021	Low-rank adapters	1.12% trainable params here
6	QLoRA [6]	2023	NF4 + paged optimizers	4B post-training on 12 GB GPU
7	ORPO [7]	2024	Reference-free preference loss	Final ORPO-v4 alignment stage
8	Qwen3.5 [8]	2026	Gated-delta hybrid 4B model	Base model; Apache-2.0

2.2 Limitations of Existing System

- Vendor runtimes (MediaPipe, Ollama) constrain the usable model set; a custom post-trained Qwen artifact needs the generality of llama.cpp/GGUF.
- A raw PEFT adapter is not phone-loadable: it must be merged with base weights and quantized before deployment.
- Stock chat samples display raw tool-call markup and hide reasoning; a tolerant parser plus an

execution loop is needed for clean agentic behaviour.

- 4-bit phone inference of a 4B model remains slow on emulators ($\approx$ tens of tokens/s) and needs $\approx$ 3 GB of device storage.

Chapter 3

Proposed System

3.1 System Overview

The system has two halves. Offline, the ORPO-v4 LoRA adapter is merged into Qwen3.5-4B and converted to F16 and then Q4_K_M GGUF with llama.cpp tooling. Online (on-device), an Android app loads the 2.5 GB GGUF through a JNI inference engine, runs a parse–execute–respond loop over the model’s `<think>` and `web_search` output, and presents thinking cards, tool cards, and final answers in a ChatGPT-style DayNight chat UI (Figure 3.1).

3.2 Components of the System

The proposed system is composed of several integrated components, each responsible for a critical stage in the post-training to on-device-answering workflow. Below is a breakdown of each major component and its role:(Figure 3.1).

1. Post-Training Pipeline

- QLoRA SFT ($r=32$, $\alpha=64$) on grounded science/math/anti-hallucination mixtures; replay continuation at low LR; ORPO preference training ($\beta=0.05$) — final ORPO-v4 adapter (≈ 190 MB).

2. Model Export Module

- PEFT `merge_and_unload` into base weights; `convert_hf_to_gguf.py` (F16, MTP draft excluded); `llama-quantize` to Q4_K_M (≈ 2.5 GB, 5.13 bits/weight).

3. Android Application

- Kotlin chat UI (auto model provisioning, streaming messages, thinking/tool cards, DayNight theme); JNI `InferenceEngine` over llama.cpp with ARM/x86_64 native libraries; in-app `WebSearch` tool (DuckDuckGo instant answers with Wikipedia fallback).

4. Tool Loop

- A tolerant parser accepts closed `<tool_call>`, unclosed, and bare `web_search({...})` forms; executes the search; feeds back `<tool_response>`; renders only the final answer (or fetched results as fallback).

Each component contributes to an end-to-end system that takes a model the team post-trained itself all the way to a usable phone assistant.

3.3 Architecture/Framework:

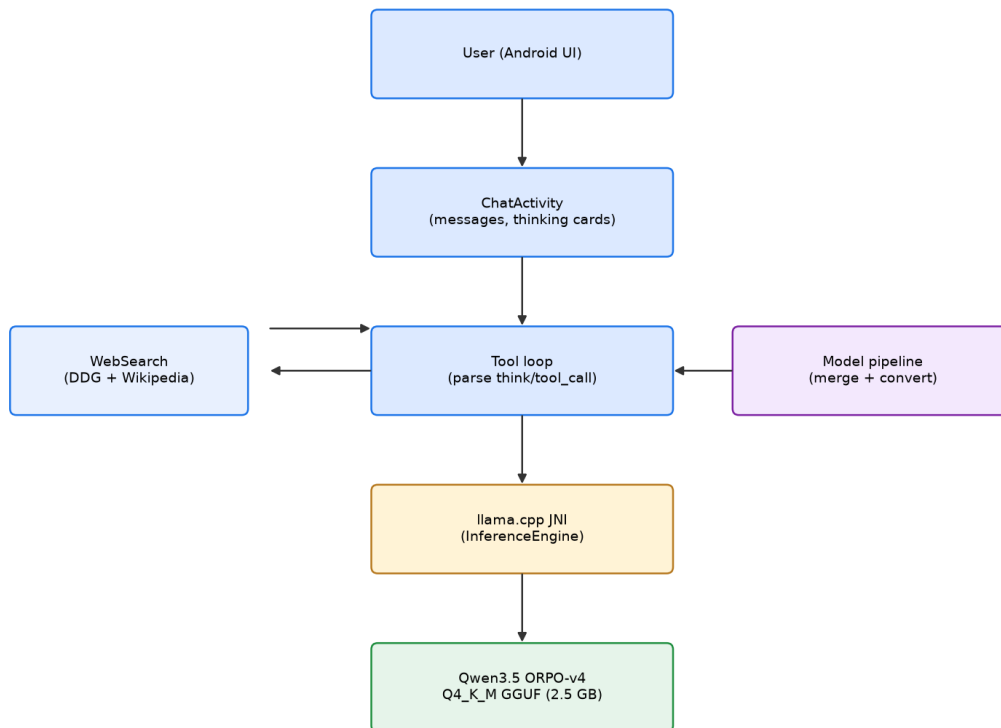


Figure 3.1: On-device system architecture

3.4 Algorithm and Process Design:

The runtime algorithm per user turn is: (1) stream generation tokens; (2) split `<think>` reasoning from body; (3) extract any `web_search` call in any of its three observed surface forms; (4) run the HTTP search tool; (5) re-prompt with the tool result plus an explicit “answer now” instruction (max two tool

rounds); (6) render thinking card, tool card, and clean answer. Markup is stripped at every stage so raw calls are never displayed.

3.5 Details of Hardware & Software

Build machine: Apple Silicon Mac, 16 GB RAM (merge/convert/quantize); training: NVIDIA RTX 4070 SUPER 12 GB. Software: Python 3.14, PyTorch/CUDA, Transformers, PEFT, TRL (ORPO), llama.cpp conversion and quantization tooling, Android Studio with SDK 36 / NDK r29 / CMake 3.31, Kotlin + Material3, minSdk 33.

Chapter 4

Design and Methodology

4.1 Design Details

The design separates model provisioning (one-time, on first launch) from conversation (every turn), and separates raw generation from presentation: the parser is the single gate between model text and the UI, which guarantees no tool markup ever reaches the screen.

4.1.1 Entity-Relationship (ER) Diagram

The ER diagram captures the chat data model: users own sessions, which contain messages; assistant messages may carry thinking traces and tool calls, and each tool call stores its fetched search results (Figure 4.1).

4.1.1.1 Key Entities and Their Attributes

- **User**
 - The person chatting with the on-device assistant. Attributes: `user_id`, `display_name`. A single-device app has exactly one active user.
- **ChatSession**
 - One conversation thread. Attributes: `session_id`, `user_id`, `started_at`. Clearing the chat starts a new session.
- **Message**
 - A single chat bubble. Attributes: `message_id`, `session_id`, `role` (user/assistant), `text`, `created_at`.
- **ThinkingTrace**

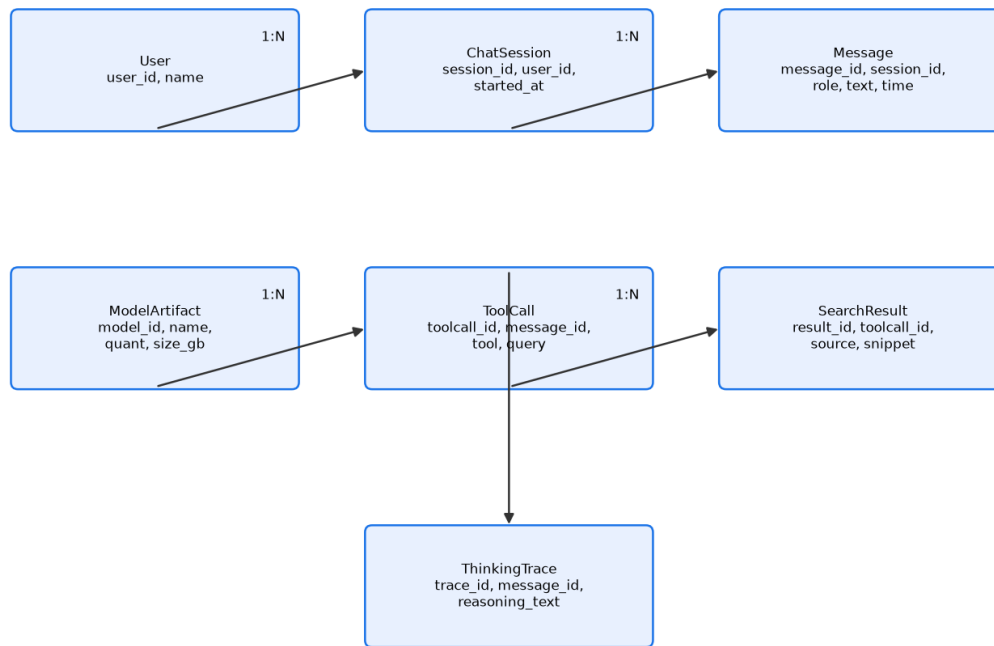


Figure 4.1: ER Diagram

- The model’s <think> reasoning for an assistant message. Attributes: `trace_id`, `message_id`, `reasoning_text`, expandable in the UI.
- **ToolCall**
 - A parsed `web_search` invocation. Attributes: `toolcall_id`, `message_id`, `tool_name`, `query`.
- **SearchResult**
 - Fetched evidence for a tool call. Attributes: `result_id`, `toolcall_id`, `source`, `snippet`.
- **ModelArtifact**
 - The deployed GGUF file. Attributes: `model_id`, `file_name`, `quant` (Q4_K_M), `size_gb` (2.5), `sha_ok`.

4.1.1.2 Relationships Between Entities

- One User owns multiple ChatSessions.
- Each ChatSession contains multiple Messages.
- Each assistant Message has at most one ThinkingTrace.
- Each assistant Message may trigger multiple ToolCalls.
- Each ToolCall stores multiple SearchResults.
- Each ChatSession uses one ModelArtifact.

4.1.1.3 Design Advantages

- **Separation of concerns:** parsing, search, and rendering are independent modules.
- **Robustness:** three tool-call surface forms are accepted; markup can never leak to the UI.
- **Modularity:** the search provider or the GGUF file can be swapped without touching the UI.
- **Traceability:** every answer links back to its thinking trace and tool evidence.

4.1.2 UML Activity Diagram

The activity flow runs from app launch and model auto-loading, through prompting, generation, parsing, optional web search, and final rendering (Figure 4.2).

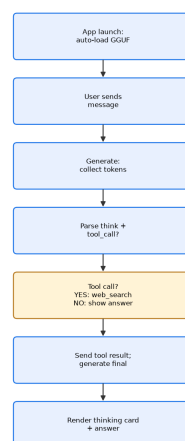


Figure 4.2: UML Activity Diagram

4.1.2.1 Activity Flow

1. App Launch and Model Load

- The provisioned Q4_K_M GGUF is auto-loaded into llama.cpp; a system prompt enables thinking and single-shot tool discipline.

2. Query Submission

- The user types in a ChatGPT-style input bar; prompts stream into the native engine.

3. Generation and Parsing

- Tokens are collected, <think> is split off, and any web_search call is extracted in closed, unclosed, or bare form.

4. Web Search Execution

- The query runs against DuckDuckGo instant answers with a Wikipedia fallback; a tool card shows what was searched.

5. Final Answer

- The tool result is fed back as <tool_response> (max two rounds) and only the clean answer is rendered; fetched results are shown directly if the model stalls.

6. Result Display

- Thinking cards (expandable), tool cards, and answer bubbles render in light or dark mode with the conversation retained across theme changes.

4.1.2.2 Features of the Activity Design

- The loop guarantees progress: every turn ends with either a model answer or the fetched web evidence, never raw markup or a spinner.

4.1.3 Data Flow Diagram (DFD) – Level 1

The Level 1 DFD shows how a prompt flows through the chat UI, the parser, the search tool, and the inference engine, grounded by the GGUF artifact and chat history stores (Figure 4.3).

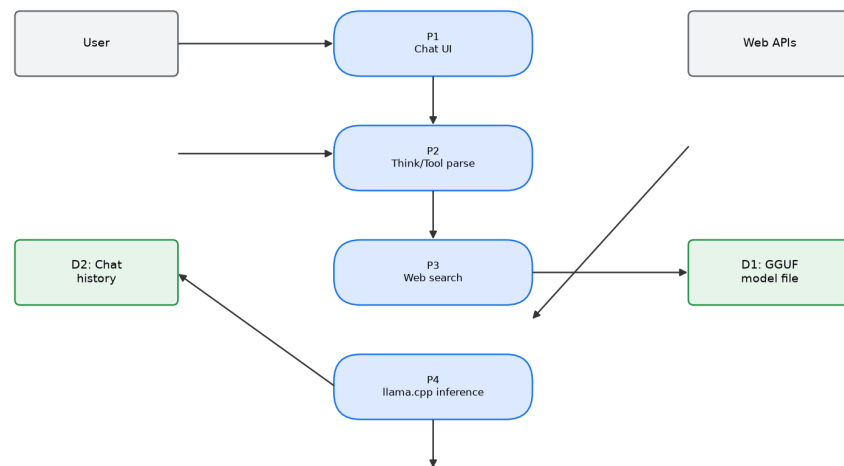


Figure 4.3: DFD Level 1

4.1.3.1 External Entity: User

The User chats through the Android UI: sending prompts and reading thinking cards, tool cards, and answers. The UI is both the entry and the exit point of every turn.

4.1.3.2 Core Processes

- **P1: Chat UI**
Renders bubbles/cards, captures input, and manages light/dark theme state.
- **P2: Think/Tool Parse**
Splits reasoning from body and extracts `web_search` calls in any observed surface form.
- **P3: Web Search**
Executes the query over HTTP (DuckDuckGo, fallback Wikipedia) and formats evidence text.
- **P4: llama.cpp Inference**
Loads the GGUF once, then prefills/decodes each turn through JNI with chat history preserved natively.

4.1.3.3 Data Stores

- D1: GGUF Model File

The 2.5 GB Q4_K_M artifact in app-private storage, loaded once per process.

- D2: Chat History

In-memory message list (retained across theme recreation), plus the native engine's conversation state.

4.1.3.4 External System: Web Search APIs

DuckDuckGo instant answers and the Wikipedia search API supply current facts without keys or accounts; the app degrades to “no results” text if both fail.

4.1.3.5 Workflow Summary

A turn starts at P1, generates at P4, parses at P2, optionally searches at P3, re-generates at P4 with the evidence, and renders at P1. All tool markup is consumed inside P2 and never reaches the screen.

4.1.3.6 Design Benefits

The loop is small, testable, and total: raw model text cannot leak past the parser, and the two-round cap plus results-fallback guarantee that every turn terminates with something useful on screen.

4.2 Methodology

1. **Post-Training (workstation/GPU):** QLoRA SFT, replay continuation, and ORPO preference training on grounded data produce the ORPO-v4 adapter.
2. **Export:** PEFT merge, GGUF F16 conversion (MTP draft excluded), and Q4_K_M quantization produce the phone artifact.
3. **App Construction:** Android Studio project on the llama.cpp Android binding: auto-load, chat UI, parser, search tool, DayNight theme.
4. **On-Device Verification:** Install on an emulator, provision the GGUF, and drive real turns (direct answers, tool-call turns, dark mode) while asserting logcat states.
5. **Evaluation:** Screening benchmarks from the training harness plus live behavioural checks (thinking shown, tool executed, clean answer rendered).

4.3 Algorithm Implementation

1. **LoRA Merge:** `PeftModel.merge_and_unload()` folds the $r=32$ adapter into base weights (text tower, matching the adapter’s training-time module names).
2. **GGUF Conversion:** `convert_hf_to_gguf.py` (F16, `--no-nextn`) maps the 426 tensors to the qwen35 architecture (32 blocks).
3. **Quantization:** `llama-quantize Q4_K_M`: 8.0 GB $\rightarrow$ 2.5 GB (5.13 bits/weight).
4. **Tool-Loop Parsing:** Regexes accept closed `<tool_call>`, unclosed, and bare `web_search({...})`; assistant-side `<tool_response>` hallucinations are stripped.
5. **Search Tool:** HTTP GET with timeouts; DuckDuckGo abstract + related topics, else Wikipedia snippets; evidence is re-fed once with an “answer now” instruction.

4.4 Details of Hardware & Software

4.4.1 Hardware Requirements

- **Processor (CPU):** ARM64 phone / Apple Silicon build host (x86_64 emulator supported)
- **RAM:** Minimum 8 GB device (Recommended: 12 GB for 4B Q4 + KV cache)
- **Storage:** At least 4 GB free (2.5 GB GGUF + app + runtime)
- **Internet:** Required only for the web-search tool (inference itself is offline)
- **GPU (Optional):** Training used an RTX 4070 SUPER 12 GB; inference is CPU

4.4.2 Software Requirements

- **Mobile OS:** Android 13+ (minSdk 33) — emulator Pixel 9, API 34 used for verification
- **IDE/SDK:** Android Studio, SDK 36, NDK r29, CMake 3.31, JDK 17
- **Language/UI:** Kotlin, Material3, DayNight theme, RecyclerView chat
- **Inference:** `llama.cpp` (GGUF, Q4_K_M, JNI binding, ARM + x86_64 ABIs)
- **Model Pipeline:** Python 3.14, PyTorch, Transformers, PEFT 0.21, TRL (ORPO), `llama.cpp` converters
- **Report Tooling:** $\text{\LaTeX}$ (Tectonic), matplotlib for charts

Chapter 5

Results and Discussions

5.1 Implementation

5.1.1 User Interface Overview

The home screen is a ChatGPT-style chat: model avatar and live status in the header, theme toggle and new-chat actions, message list, and a rounded input bar (Figure 5.1).

5.1.2 Key Features

- Auto-load: the post-trained GGUF is detected in app-private storage and loaded on launch (“Qwen3.5 ORPO-v4 ready”).
- Thinking cards: `<think>` traces render in an expandable card above each answer.
- Tool cards: every executed search is shown as a “Web search used for” card.
- Dark mode: a header toggle switches the full DayNight palette; chat survives the switch.
- New chat: one tap clears the conversation.

5.1.3 Model Loading

The 2.5 GB Q4_K_M model loads through the JNI engine in about 25–50 s on the emulator (≈ 426 tensors, qwen35 architecture, 32 blocks), after which the system prompt enables thinking and single-shot tool discipline.

5.1.4 Think/Tool Loop

Figure 5.2 shows a live turn: the question “Who wrote Romeo and Juliet?” triggered a `web_search` call, the app executed it, fed the result back, and rendered the clean final answer “William Shakespeare.” A direct question (“What is 2 plus 2?”) was answered “4” with its thinking trace and no tool call.

5.1.5 Query and Response Generation

Generation streams token by token ($\approx 25\text{--}60$ tokens/s on the emulator); the UI shows a “Thinking...” placeholder until parsing completes, then atomically renders thinking card, tool card, and answer. A two-round cap plus a show-the-evidence fallback guarantees termination.

5.2 Results

5.2.1 Training and Benchmark History

Table 5.1 condenses the full project history: the two pretraining phases that motivated the switch, and the post-training phase behind the deployed model. The 68.7M run completed all 50,000 steps yet plateaued at perplexity ≈ 98 with incoherent generations; the 200M MoE run showed a healthy loss curve but consumed its compute budget at only $\approx 13\%$ of its step target. Both results pointed the same way: under the available time and quota, post-training an existing 4B model dominates pretraining a small one.

Phase	Model	Scale	Key outcome
Pretrain dense	Dense Transformer	68.7M, 3.28B tok	val loss 4.59, PPL 98; incoherent answers
Pretrain MoE	MoE (Muon)	181M, 3.5k/26.3k steps	loss 10.3 $\rightarrow$ 5.4; stalled at $\approx 13\%$
Post-train	Qwen3.5-4B + ORPO-v4	4B + 1.1% adapter	+8.07 pp avg; GSM8K 6% $\rightarrow$ 66% (n50)

Table 5.1: Training history across project phases

Screening benchmarks from the training harness (pre-quantization, deterministic greedy decoding) are summarized in Table 5.2 and Figure 5.3: the ORPO-v4 adapter lifts the selected-metric average by +8.07 pp over base, with the largest gains on GSM8K (0% $\rightarrow$ 60% at $n=10$; 6% $\rightarrow$ 66% on the paired $n=50$ check, Figure 5.4). TruthfulQA regresses (57.38% $\rightarrow$ 43.86%), which remains open work; $n=10$ screens carry large sampling uncertainty and are reported as screening only.

Variant	MMLU-Pro CS	TruthQA	GSM8K	HellaSwag	BBH-LD5	IFEval	ARC-C
Base	0.00	57.38	0.00	70.00	30.00	22.22	30.00
ORPO-v4	30.00	43.86	60.00	50.00	20.00	22.22	40.00

Table 5.2: Seven-task screening ($n=10$ each): Base vs ORPO-v4

Measurement	Value	Where
Model file (Q4_K_M)	2.5 GB	App-private storage
Tensors / blocks	426 / 32	GGUF metadata
Model load time	25–50 s	Emulator (logcat)
Generation speed	$\approx$ 25–60 tok/s	Emulator (logcat)
Tool-call turns verified	2 (search + direct)	Live app test
Raw markup shown to user	0 cases	Parser guarantee

Table 5.3: On-device deployment measurements

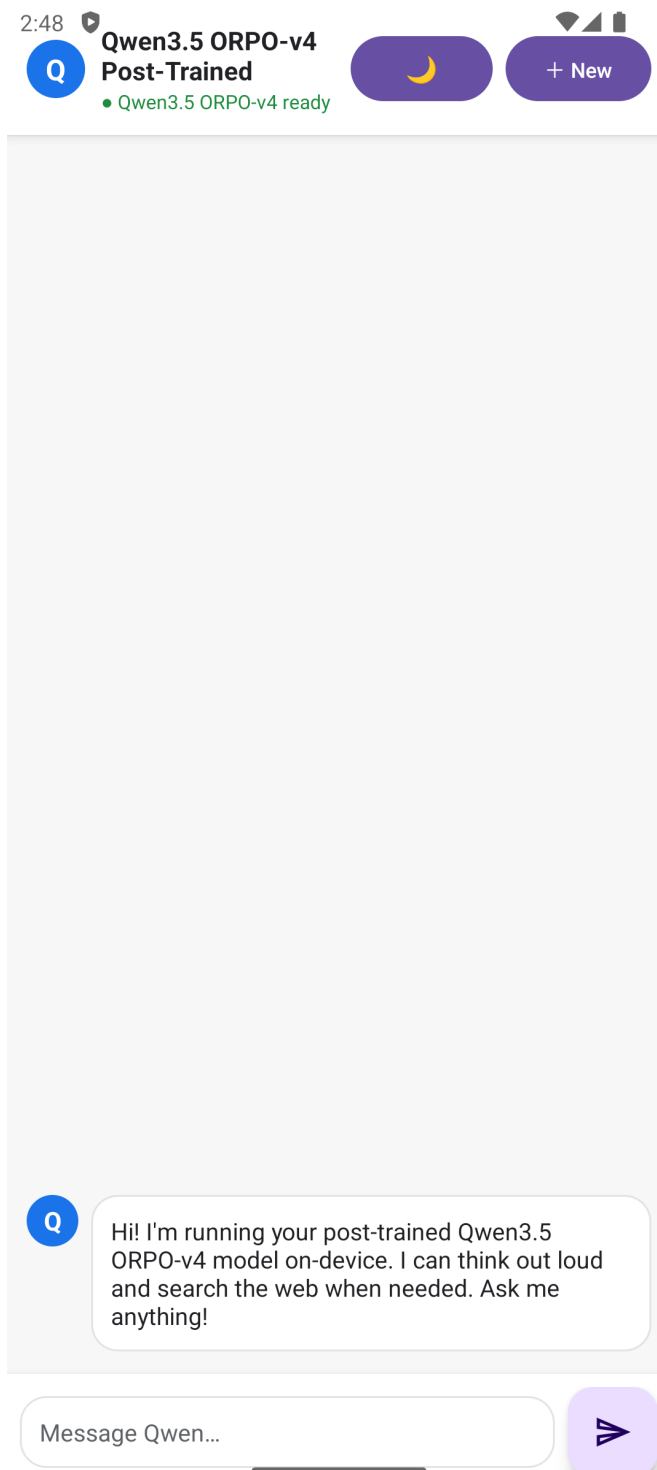


Figure 5.1: App home screen (light mode)

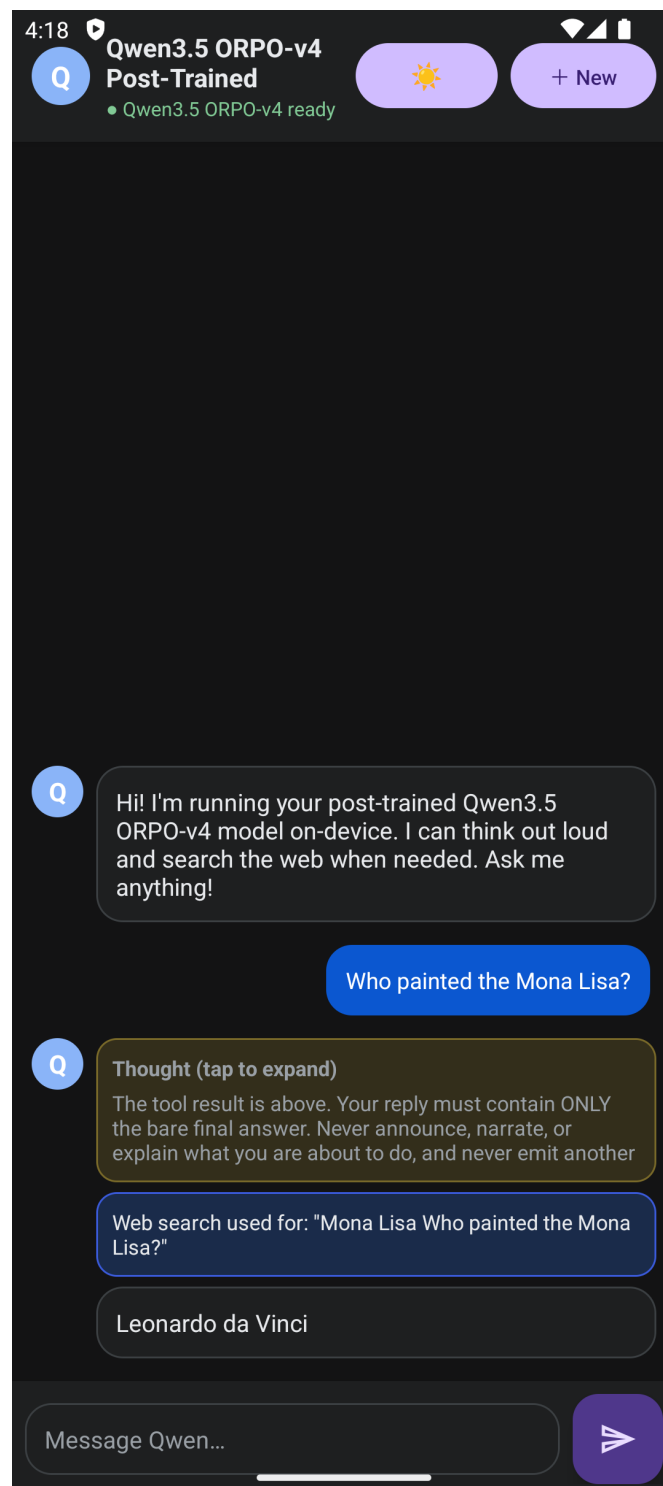


Figure 5.2: Dark-mode chat with web-search tool card and answer

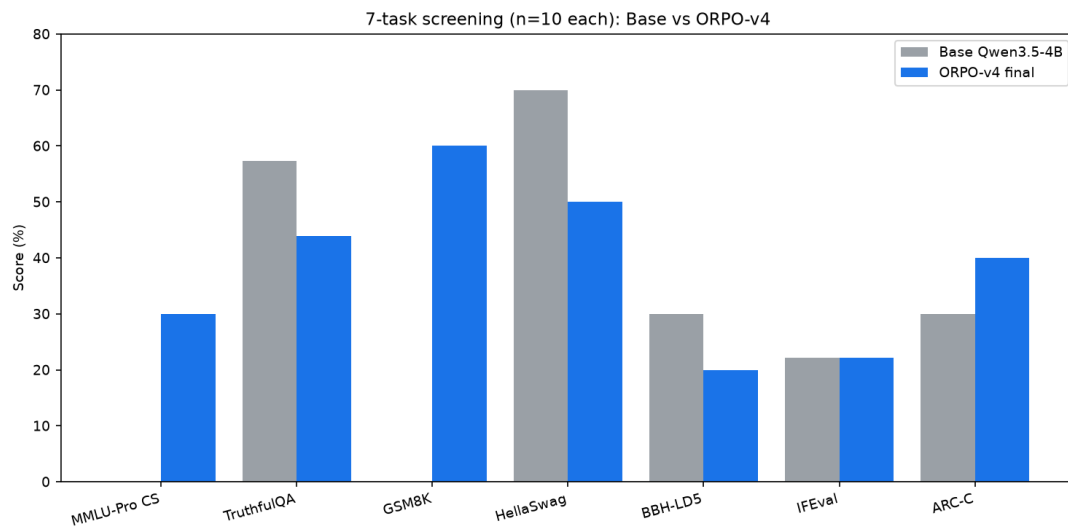


Figure 5.3: Benchmark screening: Base vs ORPO-v4

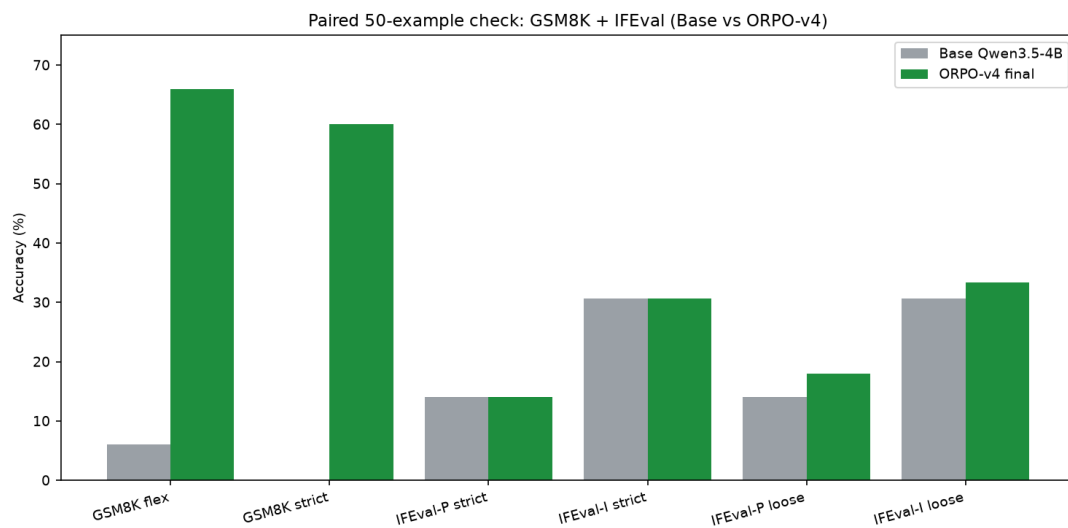


Figure 5.4: Paired GSM8K/IFEval check (n=50)

Chapter 6

Conclusion and Future Scope

This project took a self post-trained Qwen3.5-4B adapter from a LoRA checkpoint to a working Android assistant: merged and quantized to a 2.5 GB GGUF, loaded through llama.cpp, and wrapped in a polished chat UI that shows thinking, executes web-search tool calls, and never leaks raw markup. Screening shows real gains from ORPO post-training (GSM8K 6% $\rightarrow$ 66% paired), and live emulator turns confirm the full think–search–answer loop on-device. The history carries its own lesson: with student-scale compute, pretraining small models to usability is impractically slow, while post-training an existing model reaches a working assistant in days. Truthfulness regressions and the cost of 4B inference on low-end phones remain open. Future work: smaller (0.5–2B) model variants, streaming token rendering, voice input, a persistent chat database, retrieval over user documents (on-device RAG), and Play Store release hardening.

Bibliography

- [1] Gerganov, Georgi, et al. llama.cpp: LLM inference in C/C++. *GitHub repository*, 2023–2026. <https://github.com/ggml-org/llama.cpp>.
- [2] Google. MediaPipe LLM Inference API. *Developer documentation*, 2024. https://ai.google.dev/edge/mediapipe/solutions/genai/llm_inference.
- [3] MLC Team. MLC LLM: Universal LLM deployment engine. 2023. <https://llm.mlc.ai>.
- [4] Ollama Team. Ollama: Get up and running with large language models. 2023. <https://ollama.com>.
- [5] Hu, Edward J., et al. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685*, 2021.
- [6] Dettmers, Tim, et al. QLoRA: Efficient Finetuning of Quantized LLMs. *Advances in Neural Information Processing Systems* 36, 2023.
- [7] Hong, Jiwoo, et al. ORPO: Monolithic Preference Optimization without Reference Model. *arXiv preprint arXiv:2403.07691*, 2024.
- [8] Qwen Team, Alibaba Cloud. Qwen3.5: Towards Native Multimodal Agents. 2026. <https://qwen.ai>.

Plagiarism Report

The plagiarism verification report for this document will be attached here after the institutional originality check (to be added).

Publication Details / Project Achievements

Status: In preparation (to be added)

Certificates

Course and project certificates will be attached here (to be added).